

Deriving Abstract Machines

Guided Research Report

Edgard Schiebelbein

Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau, Department of Computer Science

1 Introduction

When working with programming languages and their implementations it is crucial to correctly translate specification into working code. Implementations should adhere to the specification and be free of errors. The best way to achieve this is to formally prove the correspondence between the programming language's rules and implementation. Actually doing the work and constructing proofs is often times not an easy feat. An example of such a verification is the CompCert project [Ler09] that took several years to complete.

This paper presents an approach for deriving an abstract machine from a given specification of a programming language given as evaluation rules. The focus lies on the process of starting with a simple arithmetic language and gradually adding new features. With each new extension a corresponding machine and its semantics are derived by simple reasoning about the structure of the to be implemented evaluation rules. While not being fully correct by construction, the abstract machine's correctness can be proven without much effort.

The paper is structured as follows. Section 2 presents a rather intuitive and simple approach on deriving an interpreter for a small arithmetic language. Section 3 explains the general approach of derivation used in this paper on the same language as the previous section. The next sections each describe an extension to the language, the derivation idea of the new constructs and their correctness proofs.

Everything presented is formally proven and mechanized using the proof assistant Agda [Nor09]. It is assumed the reader is familiar with Agda or at the very least with other similar tools. The entire source code can be found at <https://pl-git.informatik.uni-kl.de/hinze/calculatingabstractmachines/-/tree/master/guided-research>.

2 Deriving an Interpreter

Suppose we have a very simply language that only consists of natural numbers and addition. We are assuming we are working with an intrinsically typed language [Pie02], i.e. only type-correct terms can be constructed. Because we are only working with natural numbers, we have just one type:

```
data Type : Set where
  Nat : Type
```

Similarly, we only have one value:

```
data Value : Type → Set where
  Nat : ℕ → Value Nat
```

As mentioned before, we only have two types of terms: natural numbers of type `Nat`. The syntax is straightforward:

```
data ⊢_ : Type → Set where
  Num : ℕ → ⊢ Nat
  _⊕_ : ⊢ Nat → ⊢ Nat → ⊢ Nat
```

Where `ℕ` is the type of natural numbers defined by Agda's standard library [Nor09].

Considering the standard big-step semantics rules for such an arithmetic language we have two rules: one stating that every number evaluates to itself and one stating that the expression $e_1 \oplus e_2$ evaluates to $n_1 + n_2$ if e_1 evaluates to n_1 and e_2 to n_2 .

```
data _↓_ : ⊢ T → Value T → Set where
  NUM : ∀ n → (Num n) ↓ (Nat n)
  ADD : e₁ ↓ n₁ → e₂ ↓ n₂ → (e₁ ⊕ e₂) ↓ (n₁ + n₂)
```

Looking at the inductive definition of our semantics relation `_↓_` we can directly derive an interpreter by reading the relation `e ↓ n` as a function `eval e = n`. In the case of `NUM` it is easy to come up with what the function should do: return the underlying number.

```
eval : ⊢ T → Value T
eval (Num n) = Nat n
```

In the case of `⊕` we apply our inductive reasoning and look at the corresponding `ADD` rule. We know that we have to add the results n_1 and n_2 , as well as that we obtain those values by evaluating e_1 and e_2 . Thus, we recursively apply our `eval` function to both sub-terms and add the result.

```
eval (e₁ ⊕ e₂) = eval e₁ + eval e₂
```

With this we have successfully derived a recursive evaluation function that by looking at the semantic rules.

To make sure that our interpreter really does what it is supposed to do we can prove its behavior is correct. We do this in two steps. First we show that if our `eval` function returns some value v then there exists a proof tree showing the evaluated expression does indeed relate to the value.

```
correct-to : ∀ e → eval e ≡ v → e ↓ v
correct-to (Num x) refl = NUM x
correct-to (e₁ ⊕ e₂) refl = ADD (correct-to e₁ refl) (correct-to e₂ refl)
```

The other step is the opposite of the first. Given a proof tree showing an expression evaluates to a certain value, using `eval` results in the same value.

```
correct-from : ∀ e → e ↓ v → eval e ≡ v
correct-from (Num x) (NUM .x) = refl
correct-from (e₁ ⊕ e₂) (ADD v₁ v₂) with correct-from e₁ v₁ | correct-from e₂ v₂
... | refl | refl = refl
```

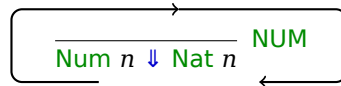
While this approach is simple and easy, it comes with a few downsides. Firstly, `eval` is not very efficient, as it is not tail recursive. Secondly, there is no way of halting evaluation and continuing afterwards.

^ Assumingly, this is where McBride's paper comes in

3 Deriving an Abstract Machine

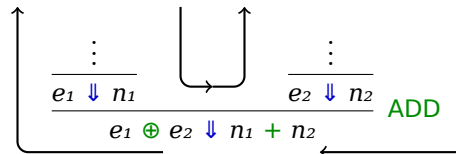
In the interpreter we constructed, an expression is evaluated recursively and in the case of addition we go further up the tree, but in no particular order. A different strategy is to traverse a proof tree in a linear fashion, with a predetermined path for each rule.

Suppose we have the same language as in Section 2. Let us look at the rules again in their tree form.

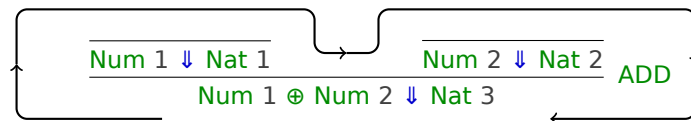


Expressions of the form **Num** n can directly be evaluated using the **NUM** rule. Traversing this tree is fairly easy by starting with the expression at the bottom, left of the evaluation relation. Following the tree up we at some point switch from evaluating to the expression being evaluated, and the resulting value is propagated back down the tree.

The case of addition requires a bit more thinking and work. In our interpreter we simply evaluated the sub-expressions recursively. This time we want to do it linearly, meaning to first evaluate the sub-expression on the left-hand side, then the one on the right-hand side. Evaluation in this case means that we go up the tree and eventually come down again with a value at our hands. This is done for both sub-expressions.



To illustrate how this process works, suppose we want to evaluate the expression **Num** $1 \oplus$ **Num** 2 . According to the **ADD** rule we have to evaluate **Num** 1 first, going up the tree. Then from the **NUM** rule we get our value **Nat** 1 , which is propagated down the tree again. The process happens to the expression **Num** 2 , i.e. first going up the tree and then propagating the value back down. In the end we get the values **Nat** 1 and **Nat** 2 and can add both numbers, resulting in **Nat** 3 . The following proof tree illustrates our approach of how the traversed path looks like.



This technique is quite intuitive, but an implementation based on it needs a few more constructions. For example, there needs to be a way of remembering values that are propagated down the tree. In the particular case of $\oplus$ we need to remember the left-hand side expression when evaluating (aka. going up the tree) the expression on the right-hand side. Also, since we evaluate from left to right, when we start evaluating the second expression we have to remember the value we got from the first one.

A solution is to use so-called *frames* [Hin20]. A frame represents a partially evaluated expression with a hole acting as a placeholder for the currently evaluating expression and lets us keep track of things that still need to be done. For our language we have two frames: one for evaluating the left sub-expression and one for the right sub-expression. In general we can notice the following: when an evaluation rule has n subtrees, then we need n frames, one for every subtree to track in which subtree we currently are.

Holes in frames can be implemented implicitly, but to make things more clear we will explicitly define what a hole is. In this case it is just a data type with a single constructor taking no arguments.

```
data Hole : Set where
  ○ : Hole
```

Now we can define the data type of a frame and both frames for our addition operation.

```
data Frame : Type → Type → Set where
  _⊕₁_ : Hole → ⊢ Nat → Frame Nat Nat
  _⊕₂_ : e ↓ n → Hole → Frame Nat Nat
```

Note the two usages of `Type` in the type signature. Since we are working with implicitly typed terms we have to keep track of types and a frame has to preserve typing. Thus, each expression of type `Frame` needs the typing information of two things: the expressions result type and the type of expression represented by the hole. In our case both are `Nat`, but when extending the language with further types and expressions we require this way of keeping track of types.

Note how both constructors represent the notion of going up and then down the expressions proof tree. In the case of `_⊕₁_` we go up the left subtree, indicated by the hole, while remembering the right expression. On the other hand for `_⊕₂_` we already have evaluated the left subtree and are currently working on the right one.

The concept of frames works nicely with simple expressions such as `Num 1 ⊕ Num 2`, but what about nested expressions? For example, consider the expression `(Num 1 ⊕ Num 2) ⊕ Num 3`. Intuitively we would create a `○ ⊕₁ Num 3` frame and start evaluating `Num 1 ⊕ Num 2` in order to know what value to plug into the hole. However, to evaluate this expression we again create a new frame, having now two of them. We need a way to keep track of them and additionally remember the order in which we created frames. Thus, we will use a *stack* of frames for this.

```
data Stack (Answer : Type) : Type → Set where
  [] : Stack Answer Answer
  _::_ : Stack Answer T₁ → Frame T₁ T₂ → Stack Answer T₂
```

The definition of `Stack` is quite standard, except for keeping track of types, making it type safe. In particular, the data type is parameterized by an answer type (which never changes) and indexed by a type that represents the topmost frame hole's type. If we want to add a new frame onto the stack, the frame's type has to match the hole's type on top. This way the stack can be viewed as a chain of sub-expression, where the expression at position n has the type required by the expression at position $n + 1$.

We now have all constructs we need to build our abstract machine. The first thing we have to define for a machine are its states. In our case this is quite easy. We already saw what two

states we have: one state for going up the proof tree and one state for going down the proof tree. Remembering that **going up means evaluating an expression** and **going down means returning a value**, we define this relation directly as a data type.

```
data State (Answer : Type) : Set where
  _↓_ : Stack Answer T → {e : ⊢ T} → e ↓ v → State Answer
  _↑_ : Stack Answer T → ⊢ T → State Answer
```

Additionally, both states need to keep track of the stack and to make state transitions type safe, **State** is parameterized by an answer type, similar to **Stack**.

Finally, we need to define the semantics of our machine. To this end we define a relation representing the machines state transitions.

```
data _→_ : State T → State T → Set where
```

Given the current state, we should be able to determine the state in which the machine should be in next. We do this by looking at the two possible states, either going down an expression's proof tree (aka. **_↓_**) or going up (aka. **_↑_**). For example, what happens when we are in the state $(stack \uparrow Num\ n)$ for some stack? Remember that going up the expressions tree corresponds to evaluating a term, meaning that in this particular case we are evaluating the expression **Num** n . And according to our **NUM** rule we get the value **Nat** n , which we return by going down the tree, i.e. going into a **_↓_** state.

```
step-Num : ∀ {n : ℕ} {stack : Stack Answer Nat}
  → (stack ↑ Num n) → (stack ↓ NUM n)
```

Now what if we are in the same state but have the expression $e_1 \oplus e_2$ to evaluate? According to how we defined to traverse the tree for such an expression we first need to evaluate e_1 . Thus, we remain in the **up** state but now with e_1 as the to be evaluated expression. However, there is one more thing: we have to remember the original expression and this is where we need our stack and frames. When transitioning to the new state we put the frame $\odot \oplus_1 e_2$ on top of the stack in order to remember from which expression we came from.

```
step-⊕ : ∀ {e1 e2 : ⊢ Nat} {stack : Stack Answer Nat}
  → (stack ↑ (e1 ⊕ e2)) → ((stack :: (⊙ ⊕₁ e₂)) ↑ e₁)
```

Since we only have two expressions, these are the only two cases for the evaluate state.

For the return state we have to think a bit about what it means to be returning a value. We already discussed how returning means having a value, or in our case a proof tree that relates an expression to a value, and propagating it back down the tree. So when we have such a value there needs to be a place to put it, since a value belongs to some expression that was evaluated. Thus, if we are in the returning state we look at the top of our stack, since it tells us where the value belongs to. In particular, if the topmost frame from the stack is a $\oplus_1$ -frame, then we know that the value belongs to the expression on the left-hand side of an addition. Now looking at how we traversed the **ADD**-tree we see that the expression on the right-hand side is evaluated next. So we switch the state from returning to evaluating and put a $\oplus_2$ -frame onto the stack with the value we have in our current state.

$\text{step-}\oplus_1 : \forall \{e_1 e_2 : \vdash \text{Nat}\} \{p_1 : e_1 \Downarrow n_1\} \{stack : \text{Stack Answer Nat}\}$
 $\rightarrow ((stack :: (\odot \oplus_1 e_2)) \downarrow p_1) \rightarrow ((stack :: (p_1 \oplus_2 \odot)) \uparrow e_2)$

The last case is a bit simpler. If there is a $\oplus_2$ -frame on top of the stack and we are returning, then we have both values need for addition. Thus, we take the frame's value, combine it if the current state's value and return the value we get by adding them both. In our case we return the combined proof tree of both.

$\text{step-}\oplus_2 : \forall \{e_1 e_2 : \vdash \text{Nat}\} \{p_1 : e_1 \Downarrow n_1\} \{p_2 : e_2 \Downarrow n_2\} \{stack : \text{Stack Answer Nat}\}$
 $\rightarrow ((stack :: (p_1 \oplus_2 \odot)) \downarrow p_2) \rightarrow (stack \downarrow \text{ADD } p_1 p_2)$

With this we have fully defined the semantics of our abstract machine. To make sure that this behavior actually corresponds to our language's semantics, we also want to proof two important properties. But before that we first define the transitive, reflexive closure of our machine's semantics relation in order to reason about sequences of operations, not just single steps.

$\text{data } \rightarrow_ : \text{State } T \rightarrow \text{State } T \rightarrow \text{Set where}$
 $\text{base} : \forall \{s : \text{State } T\} \rightarrow s \rightarrow s$
 $\text{step} : \forall \{s_1 s_2 s_3 : \text{State } T\} \rightarrow s_1 \rightarrow s_2 \rightarrow s_2 \rightarrow s_3 \rightarrow s_1 \rightarrow s_3$

For easier readable proofs we will use the following syntax. The expression $s_1 \rightarrow(p) s_2$ is a proof that state s_2 can be reached in one step using rule p from state s_1 . Similarly, $s_1 \rightarrow(p) s_2$ means that state s_2 can be reached in multiple steps p from state s_1 .

The first one states that if we evaluate an expression e with an empty stack and at some point get back a value v and an empty stack, then the term and value is in our semantic relation $e \Downarrow v$. Let us call this property *correctness*. In our case the proof for correctness is quite trivial, since we always carry the proof tree with us:

$\text{correct} : \forall (e : \vdash T) \rightarrow (v : \text{Value } T) \rightarrow (p : e \Downarrow v) \rightarrow ([\uparrow e] \rightarrow ([\downarrow p] \rightarrow e \Downarrow v)$
 $\text{correct } _ _ p _ = p$

The second property is basically the reverse of our first property: for an expression e and a value v , if $e \Downarrow v$ holds, then our abstract machine should evaluate e and return v at some point. Let us call this property *completeness*.

$\text{complete} : \forall (stack : \text{Stack Answer } T) (e : \vdash T) (v : \text{Value } T) \rightarrow (p : e \Downarrow v)$
 $\rightarrow (stack \uparrow e) \rightarrow (stack \downarrow p)$

Note how this time the stack does not need to be empty for this property to hold. Intuitively this makes sense since no matter what the stack contains, when we start evaluating a term e we do not need the stack contents, nor do we care about them in order to evaluate e . Another bonus is that since we defined *complete* for any stack, we can apply this property recursively, making our proof easier.

Since this property's proof is a bit more complex than our first one's, we do it case by case. The first case is when we want to evaluate a number and the corresponding proof tree is just our *NUM* rule. In this case the proof is simply applying our *step-Num* rule.

$\text{complete } stack (\text{Num } n) _ (\text{NUM } .n) = \text{step step-Num base}$

The second case is if we want to evaluate addition and have the corresponding proof tree for addition.

$\text{complete } stack (e_1 \oplus e_2) _ (\text{ADD } p_1 p_2) =$
 proof

```

(stack ↑ (e1 ⊕ e2))
→{ step-⊕ }
((stack :: (⊙ ⊕1 e2)) ↑ e1)
→{ complete (stack :: (⊙ ⊕1 e2)) e1 _ p1 }
((stack :: (⊙ ⊕1 e2)) ↓ p1)
→{ step-⊕1 }
((stack :: (p1 ⊕2 ⊙)) ↑ e2)
→{ complete (stack :: (p1 ⊕2 ⊙)) e2 _ p2 }
((stack :: (p1 ⊕2 ⊙)) ↓ p2)
→{ step-⊕2 }
(stack ↓ ADD p1 p2)
■

```

For this proof we look at the current state and what rule can be applied to reach the next state. From the first state we can only apply the **step-⊕** rule. In the second state we can recursively apply **complete** in order to get to the state where expression e_1 is evaluated. Then our next rule for addition can be applied and after that again a recursive call to get the state where e_2 is evaluated. Lastly we can use the last addition rule to actually get the state where we return the proof tree for adding both expressions.

Thus, we proved the derived abstract machine corresponds to the language's semantics we defined beforehand.

4 Adding Boolean Expressions

We now want to extend our language with boolean expressions, more specifically we would like to introduce branching in the form of if-expressions. Since we did almost all the groundwork in the last chapter when we defined our simple arithmetic language, extending it is quite systematic.

4.1 Syntax

The first thing we do is extending our types to support boolean expressions.

```

data Type : Set where
  Nat  : Type
  Bool : Type

```

Additionally we need to define boolean values similarly to the way we did with values of natural numbers. Again, we will be using Agdas built-in booleans as an underlying implementation.

```

data Value : Type → Set where
  Nat  : ℕ → Value Nat
  Bool : ℬ → Value Bool

```

Now we can extend our language's syntax by boolean expressions. Namely, the expressions **True**, **False** and **If_Then_Else_**.

```

data ⊢_ : Type → Set where
  Num      : ℕ → ⊢_ Nat
  ⊕_       : ⊢_ Nat → ⊢_ Nat → ⊢_ Nat
  True     : ⊢_ Bool

```

$\text{False} : \vdash \text{Bool}$
 $\text{If_Then_Else_} : \vdash \text{Bool} \rightarrow \vdash T \rightarrow \vdash T \rightarrow \vdash T$

Notice the type of `If_Then_Else_`: the first argument needs to be a boolean, however the other sub-expressions can have any arbitrary type T as long as it is the same in both branches. The entire expression is thus also of type T .

4.2 Semantics

The semantics for our new constructs are fairly simple. We do not need to change any semantic rules for arithmetic expressions, we only add new rules for boolean ones. Thus, rules that are not shown but were introduced previously are assumed to have not changed and are part of the semantics, unless stated otherwise. Additionally, we only show corresponding proof trees for the new rules, since their translation to actual code is almost a 1:1 mapping.

The rules for boolean constants are very similar to the rules for natural numbers: we simply evaluate them to their corresponding value. Additionally, since we are evaluating constants, we do not to define new stack frames, as was the case with numbers.

$\frac{}{\text{True} \Downarrow \text{true}} \text{TRUE}$
 $\frac{}{\text{False} \Downarrow \text{false}} \text{FALSE}$

The more interesting part is evaluating an if-expression, because in this case we have to worry about conditional statements value. Typically, an if-expression is evaluated according to two rules, one for when the if-condition holds and one for when it does not hold.

$\frac{e_1 \Downarrow \text{true} \quad e_2 \Downarrow v}{\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3 \Downarrow v} \text{IF}_1$
 $\frac{e_1 \Downarrow \text{false} \quad e_3 \Downarrow v}{\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3 \Downarrow v} \text{IF}_2$

4.3 Stack Frames

This is something we have not encountered yet: we need two different rules for the same expression. So how would we define our stack frames for evaluating an if-expression? If we look closely we can notice that both rules are not *completely* different. In both cases we need to evaluate e_1 and depending on the result either e_2 or e_3 is evaluated. Thus, we can extend our definition of a frame by a new constructor where we remember both e_2 and e_3 , while currently evaluating e_1 .

$\text{data Frame} : \text{Type} \rightarrow \text{Type} \rightarrow \text{Set where}$
 $\vdots$
 $\text{If}_1\text{_Then_Else_} : \text{Hole} \rightarrow \vdash T \rightarrow \vdash T \rightarrow \text{Frame } T \text{ Bool}$

Note our new frame's type `Frame T Bool`, indicating the hole being of type bool (our condition) and resulting in something of some type T , where both branches of the if-expression are of type T .

Now we need to also add frames for when e_2 and e_3 are evaluated respectively, after we are done with e_1 . For this we can leverage the fact that the frame for e_2 should only appear if $e_1 \Downarrow \text{true}$ holds. Otherwise, we would evaluate e_2 without ever needing it because we take the other branch. The same goes for e_3 and $e_1 \Downarrow \text{false}$. So we add one frame that holds a proof tree for $e_1 \Downarrow \text{true}$ as the first argument, a hole for the second argument (indicating that it is currently being evaluated) and an expression for the third argument. Additionally, we add a corresponding

frame for $e_1 \Downarrow \text{false}$ where the second argument is an expression and the third argument is a hole.

$\text{If}_2_Then_Else_ : e \Downarrow \text{true} \rightarrow \text{Hole} \rightarrow \vdash T \rightarrow \text{Frame } T T$
 $\text{If}_3_Then_Else_ : e \Downarrow \text{false} \rightarrow \vdash T \rightarrow \text{Hole} \rightarrow \text{Frame } T T$

4.4 Machine Semantics

Having defined the frames, we now need to define how our abstract machine should behave when encountering one of the new expressions. The semantics for the other expressions are not changed and the same as we defined it in Section 3. The first rule we are going to add is what happens if the machine wants to evaluate **True** or **False**. Because these expressions are values, the machine just returns the corresponding rule:

$\text{data } _ \rightarrow _ : \text{State } T \rightarrow \text{State } T \rightarrow \text{Set where}$
 $\vdots$
 $\text{step-True} : \forall \{ \text{stack} : \text{Stack Answer Bool} \} \rightarrow (\text{stack} \uparrow \text{True}) \rightarrow (\text{stack} \downarrow \text{TRUE})$
 $\text{step-False} : \forall \{ \text{stack} : \text{Stack Answer Bool} \} \rightarrow (\text{stack} \uparrow \text{False}) \rightarrow (\text{stack} \downarrow \text{FALSE})$

In order to evaluate an if-expression we first want to know if the condition does hold or not. To this end the machine is going to evaluate the first expression while adding an **If₁_Then_Else_** frame onto the stack.

$\text{step-If} : \forall \{ e_1 : \vdash \text{Bool} \} \{ e_2 e_3 : \vdash T \} \{ p_1 : e_1 \Downarrow v \} \{ \text{stack} : \text{Stack Answer } T \}$
 $\rightarrow (\text{stack} \uparrow (\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3)) \rightarrow ((\text{stack} :: (\text{If}_1 \circ \text{Then } e_2 \text{ Else } e_3)) \uparrow e_1)$

Note how the stacks topmost element needs to be of type T , since the expression **If e_1 Then e_2 Else e_3** results in a value of type T if both e_2 and e_3 are of type T . With this we have covered all new added expressions and now need to consider the cases where the machine wants to return a value and the topmost frame on its stack is one of the new ones we added.

The first case we are going to consider is the one where the topmost frame on the stack is of the form **If₁_Then_Else_** and we have a boolean value in our hands. So we know the value corresponds to the first argument of an if-expression and what we would like to do is continue with either evaluating expression e_2 or e_3 . Looking at the definitions for the corresponding frames we see that we defined them with knowledge of what value e_1 evaluates to, i.e. $e_1 \Downarrow \text{true}$ or $e_1 \Downarrow \text{false}$. Thus, in order to use these frames we need to make a distinction on what the evaluation of e_1 is. In particular, we now need *two* rules for the same frame that depend on the current value at hand, which is something we have not seen yet.

$\text{step-If}_1\text{-true} : \forall \{ e_1 : \vdash \text{Bool} \} \{ e_2 e_3 : \vdash T \} \{ p_1 : e_1 \Downarrow \text{true} \} \{ \text{stack} : \text{Stack Answer } T \}$
 $\rightarrow ((\text{stack} :: (\text{If}_1 \circ \text{Then } e_2 \text{ Else } e_3)) \downarrow p_1) \rightarrow ((\text{stack} :: (\text{If}_2 p_1 \text{ Then } \circ \text{ Else } e_3)) \uparrow e_2)$
 $\text{step-If}_1\text{-false} : \forall \{ e_1 : \vdash \text{Bool} \} \{ e_2 e_3 : \vdash T \} \{ p_1 : e_1 \Downarrow \text{false} \} \{ \text{stack} : \text{Stack Answer } T \}$
 $\rightarrow ((\text{stack} :: (\text{If}_1 \circ \text{Then } e_2 \text{ Else } e_3)) \downarrow p_1) \rightarrow ((\text{stack} :: (\text{If}_3 p_1 \text{ Then } e_2 \text{ Else } \circ)) \uparrow e_3)$

Note how the machine only knows what to do by making a case distinction on the proof tree p_1 , whereas in other cases such as in addition it does not need that much information on the current value. For the next case suppose the current frame on top of the stack is **If₂_Then_Else_** and the machine has evaluated some value v_2 for expression e_2 . We would like to return the rule

IF_1 indicating that the whole if-expression evaluates to v . But in order to apply this rule we have to restrict that e_1 evaluates to **true**.

$$\text{step-if}_2 : \forall \{e_1 : \vdash \text{Bool}\} \{e_2 e_3 : \vdash T\} \{p_1 : e_1 \Downarrow \text{true}\} \{p_2 : e_2 \Downarrow v\} \{stack : \text{Stack Answer } T\} \\ \rightarrow ((stack :: (\text{If}_2 p_1 \text{ Then } \odot \text{ Else } e_3)) \downarrow p_2) \rightarrow (stack \downarrow \text{IF}_1 p_1 p_2 e_3)$$

For the $\text{If}_3_Then_Else_$ case the approach is similar but e_1 needs to evaluate to **false**.

$$\text{step-if}_3 : \forall \{e_1 : \vdash \text{Bool}\} \{e_2 e_3 : \vdash T\} \{p_1 : e_1 \Downarrow \text{false}\} \{p_3 : e_3 \Downarrow v\} \{stack : \text{Stack Answer } T\} \\ \rightarrow ((stack :: (\text{If}_3 p_1 \text{ Then } e_2 \text{ Else } \odot)) \downarrow p_3) \rightarrow (stack \downarrow \text{IF}_2 p_1 e_2 p_3)$$

4.5 Correctness

With this we have defined how our machine should behave in every scenario that we added by introducing boolean expressions. In order to be sure our definitions are correct we once again would like to prove that two properties **correct** and **complete** still hold. Since **correct** is proven trivially, the property still holds after expanding our language and machine. On the other hand, **complete** needs to be expanded to include cases for the new expressions.

$$\text{complete} : \forall (stack : \text{Stack Answer } T) (e : \vdash T) (v : \text{Value } T) \rightarrow (p : e \Downarrow v) \\ \rightarrow (stack \uparrow e) \Rightarrow (stack \downarrow p) \\ \vdots$$

The cases for **True** and **False** are straightforward since they are values, and we have a single rule for each one.

$$\text{complete } stack \text{ True_ TRUE} = \text{step step-True base} \\ \text{complete } stack \text{ False_ FALSE} = \text{step step-False base}$$

For the case of If_Then_Else_ we actually have to make a case distinction on p since there are two possible rules that match: IF_1 and IF_2 . Both cases are similar to each other with the difference being which rules are applied depending on whether e_1 evaluates to **True** or **False**.

$$\text{complete } stack (\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3) v (\text{IF}_1 p_1 p_2 .e_3) = \\ \text{proof} \\ stack \uparrow (\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3) \\ \rightarrow (\text{step-if } \{p_1 = p_1\}) \\ (stack :: (\text{If}_1 \odot \text{ Then } e_2 \text{ Else } e_3)) \uparrow e_1 \\ \Rightarrow (\text{complete } (stack :: (\text{If}_1 \odot \text{ Then } e_2 \text{ Else } e_3)) e_1 \text{ true } p_1) \\ (stack :: (\text{If}_1 \odot \text{ Then } e_2 \text{ Else } e_3)) \downarrow p_1 \\ \rightarrow (\text{step-if}_1\text{-true}) \\ (stack :: (\text{If}_2 p_1 \text{ Then } \odot \text{ Else } e_3)) \uparrow e_2 \\ \Rightarrow (\text{complete } (stack :: (\text{If}_2 p_1 \text{ Then } \odot \text{ Else } e_3)) e_2 v p_2) \\ (stack :: (\text{If}_2 p_1 \text{ Then } \odot \text{ Else } e_3)) \downarrow p_2 \\ \rightarrow (\text{step-if}_2) \\ stack \downarrow \text{IF}_1 p_1 p_2 e_3 \\ \blacksquare \\ \text{complete } stack (\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3) v (\text{IF}_2 p_1 .e_2 p_3) = \\ \text{proof} \\ stack \uparrow (\text{If } e_1 \text{ Then } e_2 \text{ Else } e_3) \\ \rightarrow (\text{step-if } \{p_1 = p_1\}) \\ (stack :: (\text{If}_1 \odot \text{ Then } e_2 \text{ Else } e_3)) \uparrow e_1 \\ \Rightarrow (\text{complete } (stack :: (\text{If}_1 \odot \text{ Then } e_2 \text{ Else } e_3)) e_1 \text{ false } p_1)$$

```

(stack :: (lf1 ☉ Then e2 Else e3)) ↓ p1
→( step-lf1-false )
(stack :: (lf3 p1 Then e2 Else ☉)) ↑ e3
→( complete (stack :: (lf3 p1 Then e2 Else ☉)) e3 v p3 )
(stack :: (lf3 p1 Then e2 Else ☉)) ↓ p3
→( step-lf3 )
stack ↓ lf2 p1 e2 p3
■

```

5 Adding Variables

We now want to extend our language with variables, or more specifically let-expressions. To this end we need to make some changes to the definitions we used until this point.

5.1 Syntax

The main difficulty with implementing variables is keeping track of scope and handling naming of variables. Since we are working with a typed language, we need to keep track of the type a variable has. Classically this is done using a *context*, which can be seen as a mapping from variable to type. To make things simple for us, we are going to use *DeBruijn indices*[dBr72] for representing variables. The concept is quite simple: each variable is represented by a natural number indicating the distance to the term that introduced the variable. With this representation a context can simply be a sequence of types and a variables type is given by looking up the corresponding position in the context.

```

data Context : Set where
  ∅ : Context
  _:_ : Context → Type → Context

```

Using a context we can define a variable in our language as some index into the context. In order to make sure that a given type is part of the context, we will be using a witness to ensure this.

```

data _∈_ : Type → Context → Set where
  Z : T ∈ (Γ , T)
  S_ : T1 ∈ Γ → T2 ∈ (Γ , T2)

```

The relation basically defines at which position in the context we have to look for to find type T . Note how the definition of `Context` is isomorphic to that of a list and `_∈_` encodes a position in the list.

We are now ready to re-define our language's syntax using contexts. In addition to a type, an expression now also considers a context used for keeping track of variables. So instead of the unary relation $\vdash T$ for some type T , we now have the binary relation $\Gamma \vdash T$, meaning that the expression has type T under the context Γ .

```

data _⊢_ : Context → Type → Set where
  Num : ℕ → Γ ⊢ Nat
  True : Γ ⊢ Bool

```

```

False      :  $\Gamma \vdash \text{Bool}$ 
If_Then_Else :  $\Gamma \vdash \text{Bool} \rightarrow \Gamma \vdash T \rightarrow \Gamma \vdash T \rightarrow \Gamma \vdash T$ 
_⊕_        :  $\Gamma \vdash \text{Nat} \rightarrow \Gamma \vdash \text{Nat} \rightarrow \Gamma \vdash \text{Nat}$ 

```

The expressions we defined throughout the previous sections do not change a context, but we carry the context into all sub-expressions.

As mentioned before, we represent a variable as some index into the context. So if we have a witness that at a position in context Γ there is a type T , we can say we have a variable of type T under context Γ .

```

Var      :  $T \in \Gamma \rightarrow \Gamma \vdash T$ 

```

A let-expression consists of two parts: one describing the variables value as an expression and one expression which uses the variable. The important part is that the second expression has a free variable with the same type as the first expression, indicated by the `_ , _` constructor for contexts. The overall let-expression has the same type as the second expression.

```

Let_In_   :  $\Gamma \vdash T_1 \rightarrow (\Gamma, T_1) \vdash T_2 \rightarrow \Gamma \vdash T_2$ 

```

5.2 Semantics

Before defining the semantics of variables and let-expressions we first need one more definition. When evaluating a variable we require a way of retrieving the value this variable currently holds. A solution to this problem are *environments*, which map variables to their corresponding value. Environments work in similarly to contexts and we can even use our definition of contexts to define what an environment is.

```

data Environment : Context → Set where
  ∅ : Environment ∅
  _ , _ : Environment  $\Gamma \rightarrow \text{Value } T \rightarrow \text{Environment } (\Gamma, T)$ 

```

The type `Environment` is indexed by a context and has two constructors: one for the empty environment and for extending the environment. An environment is extended by providing a value of type T , which in turn also extends the context by type T . Again, one can think of this definition as simply a sequence of values with the addition of also keeping track of a sequence of types.

For looking up values in the environment we can define a function that takes an environment δ and a witness $T \in \Gamma$, i.e. a position in the context, and returns the value at this index in δ .

```

lookup : Environment  $\Gamma \rightarrow T \in \Gamma \rightarrow \text{Value } T$ 
lookup ( $\delta, x$ )  $\mathbb{Z} = x$ 
lookup ( $\delta, x$ ) ( $\mathbb{S} y$ ) = lookup  $\delta y$ 

```

With environments defined we can now continue with our new semantics. Similar to our typing relation `_ ⊢ _` that has become a binary relation due to contexts, the evaluation relation `_ ↓ _` now becomes a ternary relation `_ ⊢ _ ↓ _` due to the introduction of environments. The new relation $\delta \vdash e \Downarrow v$ states that under the environment δ the expression e evaluates to value v . Since the previously defined evaluation rules do not change the environment in any way, the rules stay the same under this new relation with minimal syntactic change. The new rules for evaluating variables and let-expressions are quite simple.

$$\frac{\delta \vdash e_1 \Downarrow v_1 \quad (\delta, v_1) \vdash e_2 \Downarrow v}{\delta \vdash \text{Let } e_1 \text{ In } e_2 \Downarrow v} \text{LET} \qquad \frac{}{\delta \vdash \text{Var } i \Downarrow \text{lookup } \delta \ i} \text{VAR}$$

When evaluating a let-expression, expression e_1 evaluates to some value v_1 under δ . Then if e_2 evaluates to v under the extended context (δ, v_1) , the whole expression evaluates to v . Intuitively this is what one would expect: we evaluate the variable's expression first and then plug its value into the second expressions free variable. When evaluating a variable we simply perform a lookup into the environment and return the value we get.

5.3 Stack Frames

In order to decide what kind of new stack frames we need, we again have to look at the structure of our evaluation rules. In the case of the VAR-rule we have no subtrees and thus do not require any new frames, similar to the case of values.

On the other hand, the LET-rule has two subtrees. So we define two new types of stack frames: one for when evaluating the left subtree and one for evaluating the right subtree.

```
data Frame : Type → Type → Set where
  ...
  Let1_In_ : Hole → (Γ , T1) ⊢ T2 → Frame T2 T1
  Let2_In_ : δ ⊢ e ⊞ v → Hole → Frame T2 T2
```

Note the argument of type (Γ , T_1) of constructor `Let1_In_`, indicating that the expression needs to have a free variable of type T_1 and the hole is representing an expression of type T_1 that is currently being evaluated.

5.4 Machine Semantics

By introducing environments, we need to modify our definition of machine states. In our current definition the constructor for going down the evaluation tree (aka. `_↓_`) carried with it a proof that an expression evaluates to a certain value: $e \Downarrow v$. However, now our relation between expressions and values additionally includes an environment. Thus, both constructor for our state definition now include environments as an additional argument.

```
data State (Answer : Type) : Set where
  _↓_ : (δ : Environment Γ) → Stack Answer T → {e : Γ ⊢ T} → δ ⊢ e ⊞ v → State Answer
  _↑_ : (δ : Environment Γ) → Stack Answer T → Γ ⊢ T → State Answer
```

Intuitively, now the abstract machine remembers in which part of the tree it is as well as the variable assignment given by the environment. Since we changed the definition of `State`, we also need to change our previous definitions of the state transition relation. For the previously defined rules we need to add an environment δ to every invocation of a constructor of `State`. These rules do not change the machine's environment, and thus no further modifications of the rules are needed.

With these changes we can now define the machine's behavior when encountering one of the new frames we added. The first two new rules are describing what to do when evaluating a variable and a let-expression. For variables, we simply return the VAR rule. For let-expressions we put the corresponding frame on the stack and continue evaluating the first sub-expression.

```

data  $\_ \rightarrow \_$  : State  $T \rightarrow$  State  $T \rightarrow$  Set where
...
step-Var :  $\forall \{v : T \in \Gamma\} \{stack : \text{Stack Answer } T\}$ 
   $\rightarrow (\delta \vdash stack \uparrow \text{Var } v) \rightarrow (\delta \vdash stack \downarrow \text{VAR } v)$ 
step-Let :  $\forall \{stack : \text{Stack Answer } T_2\}$ 
   $\rightarrow (\delta \vdash stack \uparrow (\text{Let } e_1 \text{ In } e_2)) \rightarrow (\delta \vdash (stack :: (\text{Let}_1 \odot \text{In } e_2)) \uparrow e_1)$ 

```

The more interesting rules are when returning with some value and the stack contains a let-frame at the top. When the frame is $\text{Let}_1 \odot \text{In } e_2$, we replace it with $\text{Let}_2 p_1 \text{ In } \odot$, where p_1 is the proof relating expression e_1 to value v_1 . Then we continue evaluating e_2 , and additionally extend the environment with the obtained value v_1 , such that e_2 has access to it:

```

step-Let1 :  $\forall \{e_1 : \Gamma \vdash T_1\} \{e_2 : (\Gamma, T_1) \vdash T_2\} \{p_1 : \delta \vdash e_1 \Downarrow v_1\} \{stack : \text{Stack Answer } T_2\}$ 
   $\rightarrow (\delta \vdash stack :: (\text{Let}_1 \odot \text{In } e_2) \downarrow p_1) \rightarrow ((\delta, v_1) \vdash stack :: (\text{Let}_2 p_1 \text{ In } \odot) \uparrow e_2)$ 

```

If we are in a state where we want we return p_2 and the stack contains a $\text{Let}_2 p_1 \text{ In } \odot$ on top of the stack, we take both p_1 and p_2 , and return the **LET** under the original environment δ :

```

step-Let2 :  $\forall \{e_1 : \Gamma \vdash T_1\} \{e_2 : (\Gamma, T_1) \vdash T_2\} \{p_1 : \delta \vdash e_1 \Downarrow v_1\} \{p_2 : (\delta, v_1) \vdash e_2 \Downarrow v_2\}$ 
   $\{stack : \text{Stack Answer } T_2\}$ 
   $\rightarrow ((\delta, v_1) \vdash stack :: (\text{Let}_2 p_1 \text{ In } \odot) \downarrow p_2) \rightarrow (\delta \vdash stack \downarrow \text{LET } p_1 p_2)$ 

```

Note how in the above rule the machine is in a state with environment (δ, v_1) , which is equal to the environment in the returning proof-tree p_2 .

Again, in order to come up with the machine's semantics we used the approach of looking at the new rules we introduced and traversing their trees to get step-by-step instructions on what to do when encountering a newly introduced language term.

5.5 Correctness

The proof of correctness for the new terms is similar to the other terms we introduced in previous chapters. Since we added contexts and environments, we also have to incorporate them in our correctness definitions.

In the case of **correct** nothing much changes, except that we make this property work for any environment δ :

```

correct :  $\forall (e : \Gamma \vdash T) \rightarrow (v : \text{Value } T) \rightarrow (p : \delta \vdash e \Downarrow v) \rightarrow$ 
   $(\delta \vdash [] \uparrow e) \rightarrow (\delta \vdash [] \downarrow p) \rightarrow \delta \vdash e \Downarrow v$ 
correct  $\_ \_ p \_ = p$ 

```

Similarly, in **complete** we add an arbitrary environment δ . Since the previous cases in the proof of this statement do not change the environment, we just syntactically add δ in the appropriate places and omit writing them out here:

```

complete :  $\forall (\delta : \text{Environment } \Gamma) (stack : \text{Stack Answer } T) (e : \Gamma \vdash T) (v : \text{Value } T) \rightarrow$ 
   $(p : \delta \vdash e \Downarrow v) \rightarrow (\delta \vdash stack \uparrow e) \rightarrow (\delta \vdash stack \downarrow p)$ 
  ...

```

In the case of variables we simply apply the corresponding variable rule:

`complete` δ $stack$ (`Var` e) v (`VAR` p) = `step` `step-Var` `base`

For let-expressions we the proof is straightforward. We apply the corresponding transitions and apply induction when needed:

```
complete  $\delta$   $stack$  (Let  $e_1$  In  $e_2$ )  $v$  (LET  $\{v_1 = v_1\}$   $p_1$   $p_2$ ) =
proof
  ( $\delta \vdash stack \uparrow$  (Let  $e_1$  In  $e_2$ ))
   $\rightarrow$  { step-Let }
  ( $\delta \vdash (stack :: (\text{Let}_1 \circ \text{In } e_2)) \uparrow e_1$ )
   $\rightarrow$  { complete  $\delta$  ( $stack :: (\text{Let}_1 \circ \text{In } e_2)$ )  $e_1$   $p_1$  }
  ( $\delta \vdash (stack :: (\text{Let}_1 \circ \text{In } e_2)) \downarrow p_1$ )
   $\rightarrow$  { step-Let1 }
  ( $(\delta, v_1) \vdash (stack :: (\text{Let}_2 p_1 \text{ In } \circ)) \uparrow e_2$ )
   $\rightarrow$  { complete  $(\delta, v_1)$  ( $stack :: (\text{Let}_2 p_1 \text{ In } \circ)$ )  $e_2$   $p_2$  }
  ( $(\delta, v_1) \vdash (stack :: (\text{Let}_2 p_1 \text{ In } \circ)) \downarrow p_2$ )
   $\rightarrow$  { step-Let2 }
  ( $\delta \vdash stack \downarrow \text{LET } p_1 p_2$ )
  ■
```

Note how we apply induction with an extended environment (δ, v_1) . This is possible since we defined `complete` over an arbitrary environment.

6 Adding Functions

Up until now we have extended our language with multiple base types and variables. A natural next step is to also add functions, allowing for more abstraction and re-use logic.

In this section we will focus on simple functions, which are non-recursive functions with only one argument. The restriction of only one argument simplifies our definitions and due to *currying* [Pie02] does not diminish the language's expressiveness. Recursion on the other hand makes a language more powerful. This is a topic we will tackle in the next section.

6.1 Syntax

Before defining our new syntax for functions, we have to do some preparations.

Firstly, we need a new type for functions indicating the argument type and result type. To this end we extend our definition of types, which is now becomes an inductive type:

```
data Type : Set where
  Bool : Type
  Nat : Type
   $\Rightarrow$  : Type  $\rightarrow$  Type  $\rightarrow$  Type
```

Having the defined function types, we can continue with adding new syntax for functions:

```
data  $\vdash$  : Context  $\rightarrow$  Type  $\rightarrow$  Set where
  Num      :  $\mathbb{N} \rightarrow \Gamma \vdash \text{Nat}$ 
  True     :  $\Gamma \vdash \text{Bool}$ 
  False    :  $\Gamma \vdash \text{Bool}$ 
  If_Then_Else :  $\Gamma \vdash \text{Bool} \rightarrow \Gamma \vdash T \rightarrow \Gamma \vdash T \rightarrow \Gamma \vdash T$ 
   $\oplus$       :  $\Gamma \vdash \text{Nat} \rightarrow \Gamma \vdash \text{Nat} \rightarrow \Gamma \vdash \text{Nat}$ 
```

```

Var      :  $T \in \Gamma \rightarrow \Gamma \vdash T$ 
Let_In_  :  $\Gamma \vdash T_1 \rightarrow (\Gamma, T_1) \vdash T_2 \rightarrow \Gamma \vdash T_2$ 
 $\lambda\_$     :  $(\Gamma, T_1) \vdash T_2 \rightarrow \Gamma \vdash (T_1 \Rightarrow T_2)$ 
 $\_.$       :  $\Gamma \vdash (T_1 \Rightarrow T_2) \rightarrow \Gamma \vdash T_1 \rightarrow \Gamma \vdash T_2$ 

```

The constructor $\lambda_$, also called *abstraction*, lets us create a new function of type $T_1 \Rightarrow T_2$ by applying a term with type T_2 and one free variable of type T_1 . Since we are using de Bruijn indices for a nameless representation of variables, we do not need to specify the name of the function's argument. Instead, the new variable will be represented as 0 and is increased by one each time we go deeper in a term and go over a term that introduces a new variable.

Function application $_.$ is applies a term of type T_1 to a function of type $T_1 \Rightarrow T_2$ to produce a term of type T_2 .

6.2 Semantics

With the newly introduced terms we now also have a new type of value: abstractions. Since a value is a term that can not be reduced any further, we also need to include abstractions because without an argument applied to the abstraction it can not be reduced. Typically, the value of an abstraction is called *closure* [Pie02] and includes the current environment, as well as the functions body:

```

data Value : Type → Set where
  Nat  :  $\mathbb{N} \rightarrow \text{Value Nat}$ 
  Bool :  $\mathbb{B} \rightarrow \text{Value Bool}$ 
  Closure : Environment  $\Gamma \rightarrow (\Gamma, T_1) \vdash T_2 \rightarrow \text{Value } (T_1 \Rightarrow T_2)$ 

```

The new rules for evaluating abstractions and function applications is quite standard:

$$\frac{\delta \vdash e_1 \Downarrow \text{Closure } \delta' f \quad \delta \vdash e_2 \Downarrow v_2 \quad (\delta', v_2) \vdash f \Downarrow v}{\delta \vdash e_1 . e_2 \Downarrow v} \text{ APP} \quad \frac{}{\delta \vdash \lambda e \Downarrow \text{Closure } \delta e} \text{ FUN}$$

For function application the term e_1 needs to evaluate to a closure and e_2 needs to evaluate to some value v_2 . If then the term f obtained from the closure evaluates to v under environment (δ', v_2) , the whole expression $e_1 . e_2$ evaluates to v . Intuitively, we first evaluate the function's argument and then plug the result into the function and evaluate further. This strategy is commonly known as *call-by-value* [Pie02].

The rule for abstractions simply creates a closure with the current environment and function body.

6.3 Stack Frames

As we did before, in order to derive the stack frames needed for our new syntactic structures we look at the shape of the corresponding rule's tree. In the case of **FUN** no frame is needed. On the other hand, **APP** has three subtrees: one for evaluating e_1 , one for e_2 and one for f under the extended environment. Thus, we require a total of three frames.

This is a situation we have not encountered yet. The evaluation rule of function application has only two sub-terms but three branches. In the previous constructions each constructor for a frame had as many arguments as the branches of its tree. If we try the same approach with

our first frame for function application we run into a problem: if the constructors first argument is a hole, what should the **third argument** be? A hole indicates ongoing computation, thus we do not know the value of the first argument e_1 . However, the third argument depends on the result of the first argument, since it evaluates to a closure used in the third argument. In fact, we would also need the evaluation result of e_2 for the third argument for similar reasons.

To solve this problem we stick to our notion of only one hole per stack frame but change the size of stack frames. In particular, we have one stack frame with a hole for e_1 and remember e_2 . Then we have one stack frame with the result of e_1 and a hole for e_2 . Finally, the third frame will remember both results of e_1 and e_2 , while also containing a hole representing that we are currently in the proof tree's right-most branch.

```
data Frame : Type → Type → Set where
  ...
  App1 : Hole → Γ ⊢ T1 → Frame T (T1 ⇒ T2)
  App2 : δ ⊢ e1 ↓ v1 → Hole → Frame T T1
  App3 : δ ⊢ e1 ↓ v1 → δ ⊢ e2 ↓ v2 → Hole → Frame T T1
```

6.4 Machine Semantics

With our stack frames defined, we can continue with the machine's semantics. As with previous constructions in our language, to come up with the semantics we follow the structure of the stack frames.

The first two cases are quite straightforward. Evaluating a lambda abstraction will return the **FUN** constructor with the sub-term as argument. When encountering a function application $e_1 \cdot e_2$ we continue by evaluating e_1 and add **App₁** to the stack.

```
data _→_ : State T → State T → Set where
  ...
  step-λ : ∀ {stack : Stack Answer (T1 ⇒ T2)} {e : (Γ, T1) ⊢ T2} {δ : Environment Γ}
    → (δ ⊢ stack ↑ (λ e)) → (δ ⊢ stack ↓ FUN e)
  step-App : ∀ {stack : Stack Answer T2}
    → (δ ⊢ stack ↑ (e1 · e2)) → (δ ⊢ (stack :: (App1 ○ e2)) ↑ e1)
```

The case for encountering **App₁** on top of the stack and having an evaluated value at our hands is also similar to cases we have seen before. We replace the **App₁** frame with an **App₂** frame containing our obtained value for the first term and continue with evaluating e_2 . One thing we have to ensure is that the term e_1 does actually evaluate to a closure, which we do by stating it in the type of the evaluating proof p_1 argument to the constructor.

```
step-App1 : ∀ {e1 : Γ ⊢ (T1 ⇒ T2)} {δ' : Environment Δ} {e : (Δ, T1) ⊢ T2} {e2 : Γ ⊢ T1}
  {p1 : δ ⊢ e1 ↓ (Closure δ' e)} {stack : Stack Answer T2}
  → (δ ⊢ stack :: (App1 ○ e2) ↓ p1) → (δ ⊢ stack :: (App2 p1 ○ e2) ↑ e2)
```

When encountering **App₂** on top of the stack while descending the tree we have to do a bit more work. According to the evaluation rule of function application, after evaluating both terms e_1 and e_2 we need to evaluate term e under environment δ' (which are both obtained from the closure), extended by v_2 . Thus, we exchange environment δ with δ' , v_2 , put an **App₃** frame onto the stack and evaluate e .

$$\begin{aligned} \text{step-App}_2 : & \forall \{ \text{stack} : \text{Stack Answer } T_2 \} \{ \delta' : \text{Environment } \Delta \} \{ e : (\Delta, T_1) \vdash T_2 \} \\ & \{ e_1 : \Gamma \vdash (T_1 \Rightarrow T_2) \} \{ v_2 : \text{Value } T_1 \} \{ p_1 : \delta \vdash e_1 \Downarrow (\text{Closure } \delta' e) \} \\ & \{ p_2 : \delta \vdash e_2 \Downarrow v_2 \} \\ & \rightarrow (\delta \vdash \text{stack} :: (\text{App}_2 p_1 \odot) \downarrow p_2) \rightarrow ((\delta', v_2) \vdash \text{stack} :: (\text{App}_3 p_1 p_2 \odot) \uparrow e) \end{aligned}$$

Lastly, as per the evaluation rules, having a corresponding value for e we need to switch back the original environment δ and can return the constructor for function application **APP**. Since we carry around the proof trees of all sub-terms, we also have access to this original environment. So there is no need to keep track of environments separately.

$$\begin{aligned} \text{step-App}_3 : & \forall \{ \text{stack} : \text{Stack Answer } T_2 \} \{ \delta' : \text{Environment } \Delta \} \{ e : (\Delta, T_1) \vdash T_2 \} \\ & \{ v : \text{Value } T_2 \} \{ e_1 : \Gamma \vdash (T_1 \Rightarrow T_2) \} \{ v_2 : \text{Value } T_1 \} \{ p_3 : (\delta', v_2) \vdash e \Downarrow v \} \\ & \{ p_1 : \delta \vdash e_1 \Downarrow (\text{Closure } \delta' e) \} \{ p_2 : \delta \vdash e_2 \Downarrow v_2 \} \\ & \rightarrow ((\delta', v_2) \vdash \text{stack} :: (\text{App}_3 p_1 p_2 \odot) \downarrow p_3) \rightarrow (\delta \vdash \text{stack} \downarrow (\text{APP } p_1 p_2 p_3)) \end{aligned}$$

6.5 Correctness

By now the proofs of correctness should be fairly easy, as they all follow the same structure. For values, we can directly apply the corresponding rule and are done.

$$\begin{aligned} \text{complete} : & \forall (\delta : \text{Environment } \Gamma) (\text{stack} : \text{Stack Answer } T) (e : \Gamma \vdash T) (v : \text{Value } T) \rightarrow \\ & (p : \delta \vdash e \Downarrow v) \rightarrow (\delta \vdash \text{stack} \uparrow e) \Rightarrow (\delta \vdash \text{stack} \downarrow p) \\ & \vdots \\ \text{complete } \delta \text{ stack } (\lambda e) v (\text{FUN } e) &= \text{step step-}\lambda \text{ base} \end{aligned}$$

As with other terms that contain sub-terms, the proof of correctness for function application follows the same format. First we apply the rule that initiates the evaluation. Then we can apply induction to reach a point in the evaluation where we have obtained a value and now apply the corresponding next rule. We repeat this approach until we are done.

$$\begin{aligned} \text{complete } \delta \text{ stack } (e_1 \cdot e_2) v (\text{APP } \{ \delta' = \delta' \} \{ e = e \} \{ v_2 = v_2 \} p_1 p_2 p_3) &= \\ \text{proof} & \\ (\delta \vdash \text{stack} \uparrow (e_1 \cdot e_2)) & \\ \rightarrow \{ \text{step-App} \} & \\ (\delta \vdash \text{stack} :: (\text{App}_1 \odot e_2) \uparrow e_1) & \\ \Rightarrow \{ \text{complete } \delta (\text{stack} :: (\text{App}_1 \odot e_2)) e_1 _ p_1 \} & \\ (\delta \vdash \text{stack} :: (\text{App}_1 \odot e_2) \downarrow p_1) & \\ \rightarrow \{ \text{step-App}_1 \} & \\ (\delta \vdash \text{stack} :: (\text{App}_2 p_1 \odot) \uparrow e_2) & \\ \Rightarrow \{ \text{complete } \delta (\text{stack} :: (\text{App}_2 p_1 \odot)) e_2 _ p_2 \} & \\ (\delta \vdash \text{stack} :: (\text{App}_2 p_1 \odot) \downarrow p_2) & \\ \rightarrow \{ \text{step-App}_2 \} & \\ ((\delta', v_2) \vdash \text{stack} :: \text{App}_3 p_1 p_2 \odot \uparrow e) & \\ \Rightarrow \{ \text{complete } (\delta', v_2) (\text{stack} :: (\text{App}_3 p_1 p_2 \odot)) e _ p_3 \} & \\ ((\delta', v_2) \vdash \text{stack} :: (\text{App}_3 p_1 p_2 \odot) \downarrow p_3) & \\ \rightarrow \{ \text{step-App}_3 \} & \\ (\delta \vdash \text{stack} \downarrow \text{APP } p_1 p_2 p_3) & \\ \blacksquare & \end{aligned}$$

This proof is slightly different to other proofs as we have more stack frames and need to use the induction hypothesis one additional time. Additionally, we have to be a bit careful with environments, as they change throughout the proof.

7 Adding Recursive Functions

For the final feature of our language we add recursive functions, making it Turing-complete. Recursion poses an additional challenge in the context of mechanized formalizations in proof assistants such as Agda. Since Agda only allows terminating total functions, adding recursion can potentially break things because we introduce non-termination to our language. Luckily we chose our language semantics in such a way that allows for potential infinite computation: as a relation between machine states separated by a single computation step.

7.1 Syntax

Following MINI [Hin20], recursive and non-recursive functions are syntactically different from each other. In order to use a function recursively, it must be declared as recursive when defining it.

```
data _⊢_ : Context → Type → Set where
  Num      : ℕ → Γ ⊢ Nat
  True     : Γ ⊢ Bool
  False    : Γ ⊢ Bool
  If_Then_Else : Γ ⊢ Bool → Γ ⊢ T → Γ ⊢ T → Γ ⊢ T
  ⊕        : Γ ⊢ Nat → Γ ⊢ Nat → Γ ⊢ Nat
  Var      : T ∈ Γ → Γ ⊢ T
  Let_In_   : Γ ⊢ T1 → (Γ , T1) ⊢ T2 → Γ ⊢ T2
  λ_       : (Γ , T1) ⊢ T2 → Γ ⊢ (T1 ⇒ T2)
  ·        : Γ ⊢ (T1 ⇒ T2) → Γ ⊢ T1 → Γ ⊢ T2
  LetRec_   : (Γ , T1 ⇒ T2 , T1) ⊢ T2 → Γ ⊢ (T1 ⇒ T2)
```

The constructor `LetRec` is similar to `λ` but additionally includes a function signature in the context. That way we can refer to the function and use it within the function's body.

7.2 Semantics

Before going into the evaluation rules, actually we need to add a new value. For non-recursive functions, a λ -abstraction evaluates to a closure containing the current environment and function body:

```
data Value : Type → Set where
  ...
  Closure : Environment Γ → (Γ , T1) ⊢ T2 → Value (T1 ⇒ T2)
```

We essentially save the function body, a term of type T_2 under context (Γ , T_1) . Now what would `LetRec` evaluate to? We need something similar to a closure but additionally save the function type in the context. Thus, we introduce *recursive closures*, adding the function type:

```
RecClosure : Environment Γ → (Γ , T1 ⇒ T2 , T1) ⊢ T2 → Value (T1 ⇒ T2)
```

Because we added this new value we also need two additional rules similar to those for non-recursive functions but use recursive closures:

$$\frac{\delta \vdash e_1 \Downarrow \text{RecClosure } \delta' f \quad \delta \vdash e_2 \Downarrow v_2 \quad (\delta' , \text{RecClosure } \delta' f , v_2) \vdash f \Downarrow v}{\delta \vdash e_1 \cdot e_2 \Downarrow v} \text{ RECAPP}$$

$$\frac{}{\delta \vdash \text{LetRec } e \Downarrow \text{RecClosure } \delta \ e} \text{ LETREC}$$

In the **RECAPP** rule we also add the recursive closure into the environment, to be used again in the future.

Other than this change and swapping out closures for recursive closures, both rules are essentially the same as the non-recursive cases.

7.3 Stack Frames

Following our approach for determining the amount of stack frames needed and how they should look like, we get the following:

```
data Frame : Type → Type → Set where
  ...
  RecApp1 : Hole → Γ ⊢ T1 → Frame T (T1 ⇒ T2)
  RecApp2 : δ ⊢ e1 ↓ v1 → Hole → Frame T T1
  RecApp3 : δ ⊢ e1 ↓ v1 → δ ⊢ e2 ↓ v2 → Hole → Frame T T1
```

These frames look suspiciously similar to the ones we defined in the last section. In fact, they are **exactly the same frames!** Thus, we will discard these frames and use the ones we already have, i.e. **App₁**, **App₂** and **App₃**.

7.4 Machine Semantics

Having the same frames as the last section, the machine's semantics naturally will also be similar to last's section.

As with the evaluation step for abstractions, evaluating a recursive abstraction will simply return the **LETREC** rule.

```
data _→_ : State T → State T → Set where
  ...
  step-LetRec : ∀ {stack : Stack Answer (T1 ⇒ T2)} {e : (Γ , T1 ⇒ T2 , T1) ⊢ T2}
    → (δ ⊢ stack ↑ (LetRec e)) → (δ ⊢ stack ↓ LETREC e)
```

Since function application works the same, regardless of recursive or non-recursive functions, we do not need to add a new evaluation step when starting to evaluate a recursive function application. In fact, how would we even know whether in term $e_1 \cdot e_2$ the sub-term e_1 evaluates to a recursive or non-recursive function before even evaluating it? Thus, we evaluate e_1 like we did before and only after obtaining the value it evaluates to decide how to continue. Since we already have such a rule for function application, we already did the work in the previous section get this step for free.

Similar to the situation when adding if-statements in Section 4 we now have an execution step not only depend on the kind of frame on top of the stack, but also on what e_1 evaluates to. More specifically, if we are returning and encountering **App₁** at the top of the stack, we now also add a rule stating that if the returned value is a recursive closure we continue with **App₂** and evaluating e_2 .

$$\begin{aligned} \text{step-RecApp}_1 : & \forall \{e_1 : \Gamma \vdash (T_1 \Rightarrow T_2)\} \{\delta' : \text{Environment } \Delta\} \{f : (\Delta, T_1 \Rightarrow T_2, T_1) \vdash T_2\} \\ & \{e_2 : \Gamma \vdash T_1\} \{p_1 : \delta \vdash e_1 \Downarrow (\text{RecClosure } \delta' f)\} \{stack : \text{Stack Answer } T_2\} \\ & \rightarrow (\delta \vdash stack :: (\text{App}_1 \odot e_2) \downarrow p_1) \rightarrow (\delta \vdash stack :: (\text{App}_2 p_1 \odot) \uparrow e_2) \end{aligned}$$

We can notice that both `step-App1` and `step-RecApp1` look almost identical with the difference being the environment of f and proof p_1 .

We use the same approach on both case for `App2` and `App3`. The rules for recursive functions look the same as their non-recursive counterpart with the same changes as in the above case. One detail we have to remember about the `App2` case is that we also need to include the recursive closure when evaluating the final branch. Otherwise, we would not be able to recursively call the function.

$$\begin{aligned} \text{step-RecApp}_2 : & \forall \{stack : \text{Stack Answer } T_2\} \{\delta' : \text{Environment } \Delta\} \{f : (\Delta, T_1 \Rightarrow T_2, T_1) \vdash T_2\} \\ & \{e_1 : \Gamma \vdash (T_1 \Rightarrow T_2)\} \{v_2 : \text{Value } T_1\} \{p_1 : \delta \vdash e_1 \Downarrow (\text{RecClosure } \delta' f)\} \\ & \{p_2 : \delta \vdash e_2 \Downarrow v_2\} \\ & \rightarrow (\delta \vdash stack :: (\text{App}_2 p_1 \odot) \downarrow p_2) \rightarrow \\ & \quad ((\delta', (\text{RecClosure } \delta' f), v_2) \vdash stack :: (\text{App}_3 p_1 p_2 \odot) \uparrow f) \\ \text{step-RecApp}_3 : & \forall \{stack : \text{Stack Answer } T_2\} \{\delta' : \text{Environment } \Delta\} \{f : (\Delta, T_1 \Rightarrow T_2, T_1) \vdash T_2\} \\ & \{v : \text{Value } T_2\} \{e_1 : \Gamma \vdash (T_1 \Rightarrow T_2)\} \{v_2 : \text{Value } T_1\} \{p_2 : \delta \vdash e_2 \Downarrow v_2\} \\ & \{p_3 : (\delta', (\text{RecClosure } \delta' f), v_2) \vdash f \Downarrow v\} \{p_1 : \delta \vdash e_1 \Downarrow (\text{RecClosure } \delta' f)\} \\ & \rightarrow ((\delta', (\text{RecClosure } \delta' f), v_2) \vdash stack :: (\text{App}_3 p_1 p_2 \odot) \downarrow p_3) \rightarrow \\ & \quad (\delta \vdash stack \downarrow (\text{RECAPP } p_1 p_2 p_3)) \end{aligned}$$

One important detail about this construction is that at each computation step we know what kind of function we are dealing with, since we always carry the proof about the evaluation of e_1 .

7.5 Correctness

The proof of correctness for recursive function application follows the same approach as that of if-statements. Now we have two cases when reasoning about function application: `APP` and `RECAPP`. The proof for `APP` from the previous section does not change. We only have to add a new proof for the `RECAPP` case that follows the same structure.

$$\begin{aligned} \text{complete} : & \forall (\delta : \text{Environment } \Gamma) (stack : \text{Stack Answer } T) (e : \Gamma \vdash T) (v : \text{Value } T) \rightarrow \\ & (p : \delta \vdash e \Downarrow v) \rightarrow (\delta \vdash stack \uparrow e) \Rightarrow (\delta \vdash stack \downarrow p) \\ & \vdots \\ \text{complete } \delta \text{ stack } (\text{LetRec } e) \text{ v } (\text{LETREC } p) = & \text{step step-LetRec base} \\ \text{complete } \delta \text{ stack } (e_1 \cdot e_2) \text{ v } (\text{RECAPP } \{\delta' = \delta'\} \{e = e\} \{v_2 = v_2\} p_1 p_2 p_3) = & \\ \text{proof} & \\ & (\delta \vdash stack \uparrow (e_1 \cdot e_2)) \\ \rightarrow \{ \text{step-App} \} & \\ & (\delta \vdash stack :: \text{App}_1 \odot e_2 \uparrow e_1) \\ \Rightarrow \{ \text{complete } \delta \text{ stack} :: \text{App}_1 \odot e_2 \} e_1 & (\text{RecClosure } \delta' e) p_1 \\ & (\delta \vdash stack :: \text{App}_1 \odot e_2 \downarrow p_1) \\ \rightarrow \{ \text{step-RecApp}_1 \} & \\ & (\delta \vdash stack :: \text{App}_2 p_1 \odot \uparrow e_2) \\ \Rightarrow \{ \text{complete } \delta \text{ stack} :: \text{App}_2 p_1 \odot \} e_2 _ p_2 & \\ & (\delta \vdash stack :: \text{App}_2 p_1 \odot \downarrow p_2) \\ \rightarrow \{ \text{step-RecApp}_2 \} & \\ & ((\delta', \text{RecClosure } \delta' e, v_2) \vdash stack :: \text{App}_3 p_1 p_2 \odot \uparrow e) \\ \Rightarrow \{ \text{complete } \delta', \text{RecClosure } \delta' e, v_2 \} \text{stack} :: \text{App}_3 p_1 p_2 \odot & e \text{ v } p_3 \\ & ((\delta', \text{RecClosure } \delta' e, v_2) \vdash stack :: \text{App}_3 p_1 p_2 \odot \downarrow p_3) \end{aligned}$$

$\rightarrow \{ \text{step-RecApp}_3 \}$
 $(\delta \vdash \text{stack} \downarrow \text{RECAPP } p_1 \ p_2 \ p_3)$

8 Related Work

In his work [Hut23] Hutton also derives an abstract machine from a programming language’s semantics. The difference to our approach is that Hutton starts from a recursive evaluation function and describes a process for constructing the machines operations and evaluation function. The process involves transforming the original semantics into *continuation-passing style* and then use *defunctionalization* to construct corresponding machine instructions. While this process works, the explanation of how to derive machine instructions is oftentimes not explained in detail or unintuitive, whereas the approach described in this paper follows first principles and starts from scratch.

The idea of deconstructing a term into frames and being able to stop computation at any step is similar to McBride’s work on dissecting data structures [McB08]. As a running example he uses a simple arithmetic language, turning it into a tail-recursive machine. His approach does not only focus on languages, but rather any data structure and is as such far more general than what we presented in this paper. With such a general approach the complexity also increases on how to derive these dissections. Our method is arguably simpler to understand and follow for readers not well-versed in category theory.

9 Conclusion

This paper presented an approach for deriving semantics of an abstract machine by reasoning about the source language’s semantic rules. While the constructions are **not automatically derived** and require human reasoning, the process in itself does not require complex thinking. We can distill the process into a recipe consisting of three steps:

1. Construct new frames by examining the corresponding evaluation rules. **For each subtree we define a new frame.**
2. Define new transition rules for the machine by examining what needs to happen when either a new expression is being evaluated or we are currently returning a value for a specific hole of a frame.
3. Prove the machine’s correctness using simple inductive reasoning.

There are some exceptions to these steps, e.g. in Section 4 when an expression has two distinct evaluation rules. In such cases a more manual approach is required.

A great advantage we exploit is Agda’s type system, letting us carry proofs about evaluations within a value. This makes proving correctness significantly easier.

While the process is simple, there are **optimizations to be made.** Currently, the derived machine is eagerly evaluating terms, i.e. a term is fully evaluated before continuing with another term. With **lazy evaluation** we could achieve a performance improvement by, for example, short-circuiting boolean expressions. An additional benefit would be the reduced amount of stack or making function application lazy (aka, call-by-need)

otherwise short circuiting booleans could just be implemented as syntactic sugar.
 if a && b then e1 else e2 = if a then (if b then e1) else e2
 or (if lazily) a && b = if a then b else False

frames. As an example, in the case of if-expressions we would be able to merge the two frames for evaluating the second expression into one. How?

In the future it could be feasible to extend the presented approach by incorporating such optimizations, as well as explore more programming language features and concepts.

constructors, case-elim?

References

- [dBr72] N. G de Bruijn. “Lambda Calculus Notation with Nameless Dummies, a Tool for Automatic Formula Manipulation, with Application to the Church-Rosser Theorem”. In: *Indagationes Mathematicae (Proceedings)* 75.5 (Jan. 1, 1972), pp. 381–392. ISSN: 1385-7258. DOI: 10.1016/1385-7258(72)90034-0.
- [Hin20] Ralf Hinze. *Grundlagen der Programmierung*. 2020.
- [Hut23] Graham Hutton. “Programming language semantics: It’s easy as 1,2,3”. en. In: *Journal of Functional Programming* 33 (Jan. 2023), e9. ISSN: 0956-7968, 1469-7653. DOI: 10.1017/S0956796823000072.
- [Ler09] Xavier Leroy. “Formal Verification of a Realistic Compiler”. In: *Communications of the ACM* 52.7 (July 2009), pp. 107–115. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/1538788.1538814.
- [McB08] Conor McBride. “Clowns to the Left of Me, Jokers to the Right (Pearl): Dissecting Data Structures”. In: *Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. POPL ’08. New York, NY, USA: Association for Computing Machinery, Jan. 7, 2008, pp. 287–295. ISBN: 978-1-59593-689-9. DOI: 10.1145/1328438.1328474.
- [Nor09] Ulf Norell. “Dependently Typed Programming in Agda”. In: *Advanced Functional Programming: 6th International School, AFP 2008, Heijen, The Netherlands, May 2008, Revised Lectures*. Ed. by Pieter Koopman, Rinus Plasmeijer, and Doaitse Swierstra. Lecture Notes in Computer Science. Berlin, Heidelberg: Springer, 2009, pp. 230–266. ISBN: 978-3-642-04652-0. DOI: 10.1007/978-3-642-04652-0_5.
- [Pie02] Benjamin C. Pierce. *Types and Programming Languages*. Cambridge, Mass: MIT Press, 2002. 623 pp. ISBN: 978-0-262-16209-8.